

If you observe carefully, you will notice that rsync is not preserving timestamp during the copy operation. However, we can do so by using the `-a` option as follows:

```
[root]# rsync -avh --no-W data.iso /opt/backup/
```

Now let us compare timestamps of the source and the destination files again:

```
[root]# ls -lrth
total 1.0G
-rw-r--r--. 1 root root 1.0G Dec  6 17:46 data.iso

[root]# ls -lrth /opt/backup/data.iso
-rw-r--r--. 1 root root 1.0G Dec  6 17:46 /opt/backup/
data.iso
```

This time, rsync has preserved the timestamps. The `-a` option enables the archive mode. It performs the following things:

- Enables recursive mode
- Preserves symbolic links
- Preserves permissions
- Preserves timestamp
- Preserves owner and group

f) Synchronise directory structure: Sometimes we need to create only a directory structure at the destination, which is the same as the source. rsync provides the `-d` option for this. We can use this option as follows.

Let us create a directory structure with some files:

```
[root]# mkdir -p src/java src/cpp src/php
[root]# touch src/java/Main.java
[root]# touch src/cpp/main.cpp
[root]# touch src/php/main.php
```

Now verify that the directory structure is hierarchical:

```
[root]# tree src/
src/
├── cpp
│   └── main.cpp
├── java
│   └── Main.java
└── php
    └── main.php
3 directories, 3 files
```

Let us create a directory structure at the destination using the `-d` option as follows:

```
[root]# rsync -dv src/ /opt/backup/
```

Now let us check the directory structure at the destination:

```
[root]# tree /opt/backup/
opt/backup/
├── cpp
├── java
└── php
```

3 directories, 0 files

Remote operations

So far we ran all commands on the local machine only. In this section, we will use rsync with a remote system. We can use the syntax given below when the destination is a remote system:

```
rsync [options] <source> <user>@<remote-ip>:<destination>
```

Similarly, we can use the syntax given below when the source is remote:

```
rsync [options] <user>@<remote-ip>:<destination> <source>
```

Now, let us get our hands dirty with some practical examples.

a) Dry run: It is always a good practice to do a trial run before performing actual operations. rsync has a `-n` option, which helps to do so. We can use this option as follows:

```
[root]# rsync -navz data/ root@172.16.208.11:/opt/backup/
root@172.16.208.11's password:
```

When you execute the above command, it will generate the following output:

```
sending incremental file list
created directory /opt/backup
./
data-1.iso
data-2.iso
data-3.iso

sent 133 bytes  received 62 bytes  43.33 bytes/sec
total size is 3,221,225,472  speedup is 16,519,104.98
(DRY RUN)
```

b) Synchronise files from the local to the remote server: In the previous example we verified server connectivity using a dry run. Now let us perform an actual file transfer